

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO4 – String Handling in Python

Nominal Duration: Theory 3.5 Hours / Practical 9 Hours

L.O 04: STRING HANDLING IN PYTHON

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Identify Python strings and their characteristics.
2. Create strings using single and double quotation marks.
3. Display strings using the `print()` function.
4. Assign string values to variables.
5. Create and assign multiline strings.
6. Access individual string characters using indexing.
7. Apply positive indexing to strings.
8. Slice strings to extract required characters.
9. Apply negative indexing to strings.
10. Determine string length using `len()`.
11. Apply common Python string methods.
12. Check whether characters or phrases exist within strings.
13. Concatenate two or more strings.
14. Format string output using the `format()` method.
15. Apply escape characters in Python strings.
16. Explain the immutable characteristic of Python strings.
17. Create, save, execute and test Python programs involving strings.

INFORMATION SHEET No. 04

Title: Understanding Python String Handling

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Define a Python string.
- Explain the characteristics of Python strings.
- Create strings using correct syntax.
- Display strings using `print()`.
- Assign strings to variables.
- Create multiline strings.
- Access string characters by index.
- Apply positive and negative indexing.
- Extract string portions using slicing.
- Determine string length.
- Apply common built-in string methods.
- Search for characters and phrases.
- Concatenate strings.
- Format strings using `format()`.
- Apply escape characters.
- Explain string immutability.

1. Introduction to Python Strings

A string is a sequence of characters used to represent text.

Strings are one of the most commonly used data types in Python.

Examples:

```
"Python"  
"Hello World"  
"AI Basic Python Developer"
```

A string may contain:

- Letters
- Numbers
- Spaces
- Symbols
- Punctuation

Example:

```
message = "Python 3 is easy!"
```

Check its type:

```
print(type(message))
```

Output:

```
<class 'str'>
```

The Python data type for strings is:

```
str
```

2. Creating Python Strings

Python strings can be created using either single quotation marks or double quotation marks.

Single Quotes

```
name = 'Dara'
```

Double Quotes

```
name = "Dara"
```

Both represent strings.

Example:

```
course1 = 'Python'
course2 = "Python"

print(course1)
print(course2)
```

Output:

```
Python
Python
```

3. Displaying Strings with print()

The print() function displays information on the screen.

Syntax:

```
print("text")
```

Example:

```
print("Hello Python")
```

Output:

```
Hello Python
```

Another example:

```
print("AI Basic Python Developer")
```

Output:

AI Basic Python Developer

4. Assigning a String to a Variable

A string can be stored in a variable.

Example:

```
name = "Dara"
```

Here:

- name is the variable.
- "Dara" is the string value.

Display the value:

```
print(name)
```

Output:

Dara

Another example:

```
course = "Python Programming"
institution = "Technical Training Center"

print(course)
print(institution)
```

5. Multiline Strings

A multiline string contains text across several lines.

Python supports multiline strings using triple quotation marks.

Triple Double Quotes

```
message = """Welcome to Python.
This course teaches Python programming.
Practice is important."""

print(message)
```

Triple Single Quotes

```
message = '''Welcome to Python.
Learn Python syntax.
Practice Python programming.'''

print(message)
```

Output preserves the line structure.

6. Strings as Sequences of Characters

A Python string is a sequence of characters.

Consider:

Python

The characters have positions called indexes.

Character	P	y	t	h	o	n
Positive Index	0	1	2	3	4	5
Negative Index	-6	-5	-4	-3	-2	-1

Python indexing begins at:

0

Therefore, the first character of "Python" is at index 0.

7. Accessing Characters Using Indexing

Individual characters can be accessed using square brackets.

Syntax:

```
string[index]
```

Example:

```
word = "Python"

print(word[0])
```

Output:

P

Example:

```
print(word[1])
```

Output:

y

Example:

```
print(word[5])
```

Output:

n

8. Character Position as an Array-Like Sequence

Although a Python string is not a list or array, its characters can be accessed using indexes in an array-like manner.

Example:

```
course = "Python"

print(course[0])
print(course[1])
print(course[2])
```

Output:

```
P
y
t
```

The important rule is:

The first character has index 0.

9. String Slicing

Slicing extracts part of a string.

Syntax:

```
string[start:end]
```

The start position is included.

The end position is excluded.

Example:

```
word = "Python"

print(word[0:3])
```

Output:

```
Pyt
```

Indexes:

```
  P y t h o n
  0 1 2 3 4 5
```

word[0:3] retrieves indexes:

```
0, 1, 2
```

10. Slicing from the Beginning

The starting index can be omitted.

Example:

```
word = "Python"

print(word[:3])
```

Output:

Pyt

This means:

Start from the beginning and stop before index 3.

11. Slicing to the End

The ending index can also be omitted.

Example:

```
word = "Python"

print(word[2:])
```

Output:

thon

This means:

Start at index 2 and continue to the end.

12. Slicing with Step

Python also supports a step value.

Syntax:

```
string[start:end:step]
```

Example:

```
word = "Python"

print(word[0:6:2])
```

Output:

Pto

This selects every second character.

13. Negative Indexing

Negative indexing counts characters from the end of the string.

The last character has index:

-1

Example:

```
word = "Python"
print(word[-1])
```

Output:

n

Example:

```
print(word[-2])
```

Output:

o

For "Python":

Character	P	y	t	h	o	n
Negative Index	-6	-5	-4	-3	-2	-1

14. Negative Slicing

Negative indexes can also be used in slicing.

Example:

```
word = "Python"
print(word[-4:-1])
```

Output:

tho

This starts at index -4 and stops before index -1.

15. Determining String Length

The built-in `len()` function determines the number of characters in a string.

Syntax:

```
len(string)
```

Example:

```
word = "Python"
print(len(word))
```

Output:

6

Spaces are also counted.

Example:

```
message = "Hello Python"

print(len(message))
```

Output:

12

The space between Hello and Python counts as one character.

16. Python String Methods

Python provides many built-in methods for working with strings.

Common methods include:

Method	Purpose
upper()	Converts text to uppercase
lower()	Converts text to lowercase
capitalize()	Capitalizes first character
title()	Capitalizes words
strip()	Removes leading/trailing whitespace
replace()	Replaces text
split()	Splits a string
find()	Finds position of text
count()	Counts occurrences
startswith()	Checks beginning
endswith()	Checks ending

17. upper() Method

The upper() method converts letters to uppercase.

Example:

```
course = "python"

print(course.upper())
```

Output:

PYTHON

18. lower() Method

The lower() method converts letters to lowercase.

Example:

```
course = "PYTHON"  
  
print(course.lower())
```

Output:

```
python
```

19. capitalize() Method

The `capitalize()` method converts the first character to uppercase.

Example:

```
text = "python programming"  
  
print(text.capitalize())
```

Output:

```
Python programming
```

20. title() Method

The `title()` method capitalizes the first letter of each word.

Example:

```
text = "python programming language"  
  
print(text.title())
```

Output:

```
Python Programming Language
```

21. strip() Method

The `strip()` method removes whitespace from the beginning and end.

Example:

```
name = "  Dara  "  
  
print(name.strip())
```

Output:

```
Dara
```

22. replace() Method

The `replace()` method replaces specified text.

Example:

```
message = "I like Java"

print(message.replace("Java", "Python"))
```

Output:

```
I like Python
```

23. split() Method

The `split()` method separates a string into parts.

Example:

```
message = "Python is easy"

print(message.split())
```

Output:

```
['Python', 'is', 'easy']
```

A separator can also be specified.

Example:

```
data = "Dara,20,Python"

print(data.split(","))
```

Output:

```
['Dara', '20', 'Python']
```

24. find() Method

The `find()` method searches for text and returns its starting position.

Example:

```
message = "I like Python"

print(message.find("Python"))
```

Output:

```
7
```

If the text is not found, `find()` returns:

```
-1
```

25. Checking a Phrase or Character in a String

Python provides the membership operators:

```
in
not in
```

Example:

```
message = "Python is easy"

print("Python" in message)
```

Output:

True

Example:

```
print("Java" in message)
```

Output:

False

26. Checking a Character

A single character can also be checked.

Example:

```
word = "Python"

print("P" in word)
```

Output:

True

Example:

```
print("z" in word)
```

Output:

False

27. Using not in

The not in operator determines whether text does not exist in a string.

Example:

```
message = "Python Programming"

print("Java" not in message)
```

Output:

True

28. String Concatenation

Concatenation means joining strings together.

The + operator can concatenate strings.

Example:

```
first_name = "Dara"
last_name = "Sok"

full_name = first_name + last_name

print(full_name)
```

Output:

DaraSok

To include a space:

```
full_name = first_name + " " + last_name

print(full_name)
```

Output:

Dara Sok

29. Required Concatenation Exercise

Assign:

```
s = "favo"
t = "rite"
```

Concatenate s and t.

Example:

```
s = "favo"
t = "rite"

result = s + t

print(result)
```

Output:

favorite

This directly addresses Knowledge Requirement 1.13.

30. String Formatting

String formatting allows values to be inserted into strings.

Python provides the `format()` method.

Example:

```

name = "Dara"
age = 20

message = "My name is {} and I am {} years old.".format(name, age)

print(message)

```

Output:

My name is Dara and I am 20 years old.

31. format() with Numbered Placeholders

Placeholders can use indexes.

Example:

```

name = "Dara"
course = "Python"

message = "Student {0} studies {1}.".format(name, course)

print(message)

```

Output:

Student Dara studies Python.

32. format() with Named Placeholders

Named placeholders can also be used.

Example:

```

message = "Student {name} studies {course}.".format(
    name="Dara",
    course="Python"
)

print(message)

```

Output:

Student Dara studies Python.

33. Escape Characters

An escape character allows special characters to be included inside strings.

Escape sequences begin with:

\

Common escape sequences include:

Escape Sequence	Purpose
-----------------	---------

Escape Sequence	Purpose
\n	New line
\t	Tab
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark

34. New Line —

Example:

```
print("Python\nProgramming")
```

Output:

```
Python
Programming
```

35. Tab — xample:

```
print("Name\tAge\tCourse")
print("Dara\t20\tPython")
```

Output resembles:

```
Name    Age    Course
Dara    20     Python
```

36. Quotation Marks in Strings

Example:

```
print("He said, \"Python is easy.\")
```

Output:

```
He said, "Python is easy."
```

37. Backslash

To display a backslash, use:

```
\\
```

Example:

```
print("C:\\Python\\Projects")
```

Output:

```
C:\Python\Projects
```

38. Python Strings are Immutable

Python strings are immutable.

Immutable means that the individual characters of an existing string cannot be changed directly.

Example:

```
word = "Python"
```

The following is invalid:

```
word[0] = "J"
```

Python reports an error because strings cannot be modified by assigning a new value to an individual character.

Instead, create a new string.

Example:

```
word = "Python"
new_word = "J" + word[1:]
print(new_word)
```

Output:

```
Jython
```

39. Complete String Handling Example

```
course = "Python Programming"

print("Original:", course)
print("Length:", len(course))
print("First Character:", course[0])
print("Last Character:", course[-1])
print("Slice:", course[0:6])
print("Upper:", course.upper())
print("Lower:", course.lower())
print("Contains Python:", "Python" in course)

s = "favo"
t = "rite"
print("Concatenation:", s + t)

name = "Dara"
age = 20

print("My name is {} and I am {} years old.".format(name, age))

print("Python\nProgramming")
```

This program demonstrates the major string-handling concepts required in LO4.

SELF-CHECK No. 4.1

Part A: Multiple Choice

1. Which Python data type represents text?

- A. int B. float C. str D. bool

2. Which function displays a string?

- A. display() B. show() C. print() D. output()

3. Which is a valid string assignment?

- A. name = Dara B. name = "Dara"
C. "name" = Dara D. name :=: Dara

4. Which can create a multiline string?

- A. Triple quotation marks B. Parentheses only
C. Square brackets D. Curly brackets

5. What is the first index of a Python string?

- A. -1 B. 0 C. 1 D. 2

6. Given word = "Python", what is word[0]?

- A. P B. y C. n D. Python

7. What does word[0:3] return when word = "Python"?

- A. Py B. Pyt C. Pyth D. Python

8. Which index retrieves the last character?

- A. 0 B. 1 C. -1 D. -2

9. Which function determines string length?

- A. size() B. length() C. count() D. len()

10. Which method converts a string to uppercase?

- A. upper() B. uppercase() C. big() D. capital()

11. Which operator checks whether text exists in a string?

- A. = B. in C. + D. is

12. Which operator concatenates strings?

- A. - B. / C. + D. %

13. Which method is required in this LO for string formatting?

- A. format() B. print() C. type() D. input()

14. Which escape sequence creates a new line?

- A. \t B. \n C. \\ D. \"

15. Python strings are:

- A. Mutable B. Immutable C. Numeric only D. Boolean only

ANSWER KEY No. 4.1

Part A

No.	Answer
1	C
2	C
3	B
4	A
5	B
6	A
7	B
8	C
9	D
10	A
11	B
12	C
13	A
14	B
15	B

OPERATION SHEET No. 4.1

Title: Create, Access and Manipulate Python Strings

Performance Objective

Given a computer with Python installed and Python IDLE or another approved development environment, the trainee must create strings, access their characters, slice strings, use string methods, concatenate and format strings, apply escape characters, save the program and verify its output.

Description

This operation demonstrates the basic procedures required to create and manipulate Python string data.

Picture/Overview



Figure 1: Python Strings

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/IDE	Software	1

No.	Description	Unit	Qty
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Book	Copy	1
7	Programming Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDLE/IDE	Development environment opens successfully
2	Create a Python source file	New program file is created
3	Create a string	Correct quotation marks are used
4	Assign string to variable	String value is correctly stored
5	Display string	print() produces correct output
6	Create multiline string	Triple quotes are correctly used
7	Access characters	Correct indexes are used
8	Slice string	Required substring is extracted
9	Apply negative indexing	Correct character is retrieved
10	Determine string length	len() returns correct length
11	Apply string methods	Required transformations are correct
12	Search string	Phrase/character is correctly checked
13	Concatenate strings	Strings are joined correctly
14	Apply format()	Values appear in correct placeholders
15	Apply escape characters	Required formatted output is displayed
16	Save program	.py extension is used
17	Execute program	Program runs successfully
18	Verify output	Actual output matches requirements

Operation Procedure

Step 1 — Create a String

```
course = "Python Programming"
```

Step 2 — Display the String

```
print(course)
```

Step 3 — Create a Multiline String

```
description = """Python is a programming language.
It is easy to learn.
It is widely used."""
print(description)
```

Step 4 — Access Characters

```
print(course[0])
print(course[1])
print(course[-1])
```

Step 5 — Slice the String

```
print(course[0:6])
print(course[:6])
print(course[7:])
```

Step 6 — Determine Length

```
print(len(course))
```

Step 7 — Apply Methods

```
print(course.upper())
print(course.lower())
print(course.replace("Python", "AI"))
```

Step 8 — Search

```
print("Python" in course)
print("Java" not in course)
```

Step 9 — Concatenate

```
s = "favo"
t = "rite"
print(s + t)
```

Step 10 — Format String

```
name = "Dara"
age = 20

print("My name is {} and I am {} years old.".format(name, age))
```

Step 11 — Apply Escape Characters

```
print("Name\tAge\tCourse")
print("Dara\t20\tPython")
print("Python\nProgramming")
```

Step 12 — Save

Save as:

```
string_handling.py
```

Step 13 — Execute and Verify

Run the program and compare actual results with the expected output.

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 4.2

Title: Create, Display and Access Python Strings

Performance Task

Create Python strings, assign them to variables, display them and access individual characters using positive and negative indexing.

Performance Objective

Given Python IDLE/IDE and programming requirements, the trainee must create single-line and multiline strings and retrieve required characters using correct indexes.

Description

The trainee will practice basic string creation, display, multiline strings and character indexing.

Picture/Overview

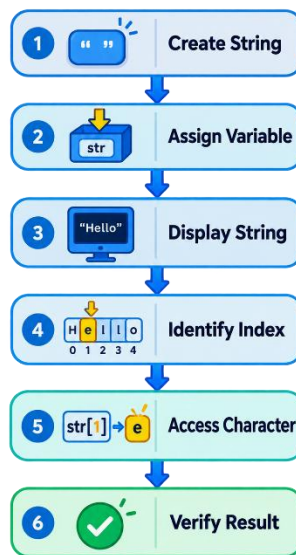


Figure 2: Task Sheet 1

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDLE/IDE	Software	1
4	String Exercise File	Set	1
5	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDE	IDE opens successfully
2	Create string variable	Correct syntax is used
3	Display string	Correct output is produced

No.	Procedure/Step	Key Performance
4	Create multiline string	Triple quotes are used
5	Access first character	Index 0 is correctly applied
6	Access other characters	Correct positive indexes are used
7	Access last character	Negative index -1 is applied
8	Save and execute	Program runs correctly

Practical Exercise

Create:

```
course = "Python"
```

Display:

```
print(course)
```

Create:

```
message = """Welcome to Python.
Learn string handling.
Practice Python programming."""
```

Display the characters at:

```
Index 0
Index 1
Index 3
Index -1
Index -2
```

Complete:

Expression	Result
course[0]	_____
course[1]	_____
course[3]	_____
course[-1]	_____
course[-2]	_____

Assessment Method

- Direct observation
- Practical demonstration
- Worksheet review
- Output verification

TASK SHEET No. 4.3

Title: Apply String Slicing and Determine String Length

Performance Task

Extract required portions of strings using slicing and determine the number of characters in a string.

Performance Objective

Given a Python string and required character ranges, the trainee must correctly apply positive and negative slicing and use `len()` to determine string length.

Description

Use:

```
course = "Python Programming"
```

to practice string slicing.

Picture/Overview

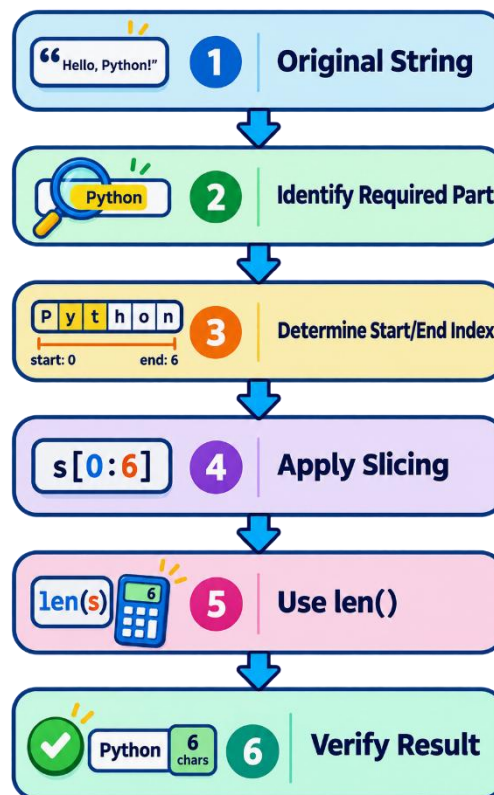


Figure 3: Apply String Slicing

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create string	Correct value is assigned
2	Identify indexes	Required positions are determined
3	Apply positive slicing	Correct substring is extracted
4	Slice from beginning	Omitted start index is correctly used
5	Slice to end	Omitted end index is correctly used
6	Apply negative slicing	Required substring is extracted
7	Use len()	Correct length is returned
8	Verify output	Results match requirements

Practical Exercise

```
course = "Python Programming"
```

```
print(course[0:6])
print(course[:6])
print(course[7:])
print(course[-11:])
print(len(course))
```

Complete:

Expression	Result
course[0:6]	_____
course[:6]	_____
course[7:]	_____
course[-11:]	_____
len(course)	_____

Assessment Method

- Direct observation
- Practical exercise
- Worksheet review
- Output verification

TASK SHEET No. 4.4

Title: Apply Python String Methods and Search Strings

Performance Task

Use built-in string methods to manipulate string values and search for characters and phrases.

Performance Objective

Given a Python string, the trainee must apply common string methods and determine whether specified characters or phrases exist within the string.

Description

The trainee will practice:

upper()
lower()
capitalize()
title()
strip()

replace()
split()
find()
in
not in

Picture/Overview

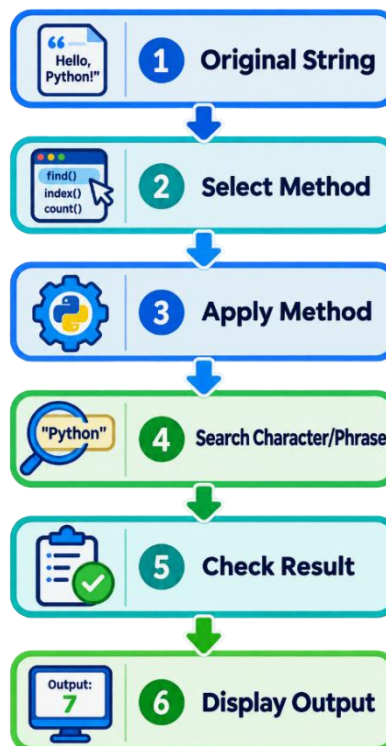


Figure 4: Apply Python String Methods

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Programming Worksheet	Copy	1
5	Python Reference Book	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create string	String is assigned correctly
2	Apply upper()	Uppercase result is correct
3	Apply lower()	Lowercase result is correct
4	Apply title()	Title-case result is correct
5	Apply strip()	Leading/trailing spaces are removed
6	Apply replace()	Specified text is replaced
7	Apply split()	String is separated correctly
8	Apply find()	Correct position is returned
9	Apply in	Presence is correctly determined
10	Apply not in	Absence is correctly determined
11	Verify output	Results are correct

Practical Exercise

```
message = " Python Programming is Easy "
```

```
print(message.upper())
print(message.lower())
print(message.title())
print(message.strip())
print(message.replace("Easy", "Powerful"))
print(message.split())
print(message.find("Python"))
print("Python" in message)
print("Java" not in message)
```

Record your results.

Operation	Result
upper()	_____
lower()	_____
title()	_____
strip()	_____
replace()	_____
split()	_____
find("Python")	_____
"Python" in message	_____
"Java" not in message	_____

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Output verification

TASK SHEET No. 4.5

Title: Concatenate and Format Python Strings

Performance Task

Combine strings using concatenation and create formatted output using the `format()` method.

Performance Objective

Given several string variables and values, the trainee must concatenate strings and correctly insert values into formatted strings using `format()`.

Description

The trainee will practice the `+` operator and `format()` method.

Picture/Overview

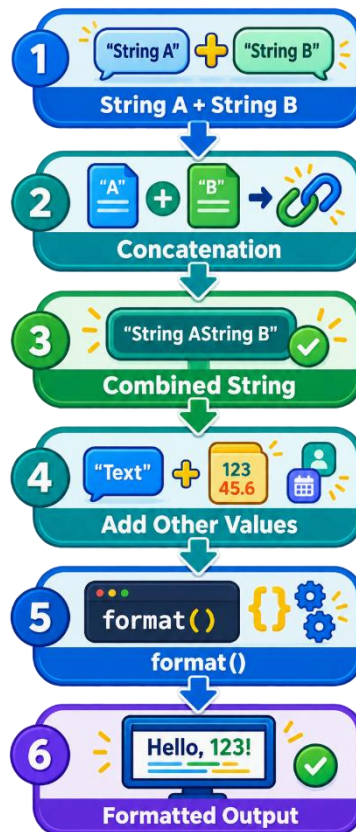


Figure 5: Concatenate and Format Python Strings

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create string variables	Values are assigned correctly
2	Apply +	Strings are concatenated correctly
3	dd spaces where required	Output is readable
4	Create format string	Placeholders are correct
5	Apply format()	Values appear in correct positions
6	Display output	Correct string is displayed
7	Save and execute	Program runs successfully

Practical Exercise A

Assign:

```
s = "favo"
t = "rite"
```

Concatenate and display the result.

Expected:

```
favorite
```

Practical Exercise B

Create:

```
first_name = "Dara"
last_name = "Sok"
```

Produce:

```
Dara Sok
```

Practical Exercise C

Given:

```
name = "Dara"
age = 20
course = "Python"
```

Use format() to produce:

```
My name is Dara. I am 20 years old and I study Python.
```

Assessment Method

- Practical demonstration
- Direct observation
- Source-code inspection
- Output verification

TASK SHEET No. 4.6

Title: Implement Escape Characters in Python Strings

Performance Task

Apply Python escape characters to produce formatted text output.

Performance Objective

Given text-output requirements, the trainee must correctly apply newline, tab, quotation-mark and backslash escape sequences.

Description

The trainee will use:

```
\n
\t
\"
\'
\\
```

Picture/Overview

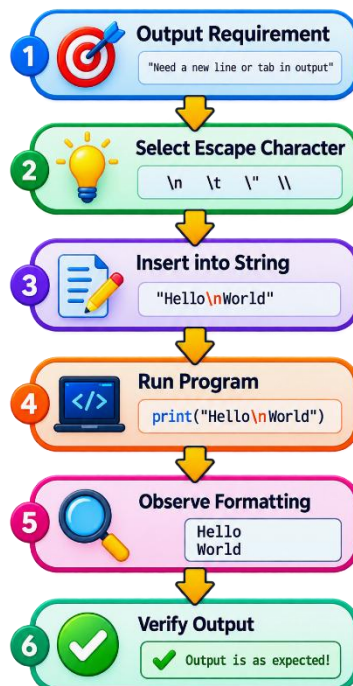


Figure 6: Implement Escape Characters in Python Strings

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	1. Identify output requirement	Appropriate escape sequence is selected
2	2. Apply \n	New line is correctly displayed
3	3. Apply \t	Tab spacing is displayed
4	4. Apply \" or \'	Quotation mark is displayed correctly
5	5. Apply \\	Backslash is displayed correctly
6	6. Execute program	Program runs successfully
7	7. Verify output	Formatting meets requirements

Practical Exercise

Create output using:

```
print("Python\nProgramming")
print("Name\tAge\tCourse")
print("Dara\t20\tPython")
print("He said, \"Python is easy.\")
print("C:\\Python\\Projects")
```

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

JOB SHEET No. 4.1

Title: Develop a Student String Information Processing Program

Performance Task

Develop a complete Python program demonstrating the required string-handling operations.

Performance Objective

Given a computer, Python IDLE/IDE and program requirements, the trainee must create, save, execute, test and verify a Python program that demonstrates string declaration, indexing, slicing, string methods, searching, concatenation, formatting and escape characters.

Description

Develop a program named:

`student_string_profile.py`

The program will create and process student and course information using Python strings. The program must demonstrate all major LO4 string-handling competencies.

Picture/Overview

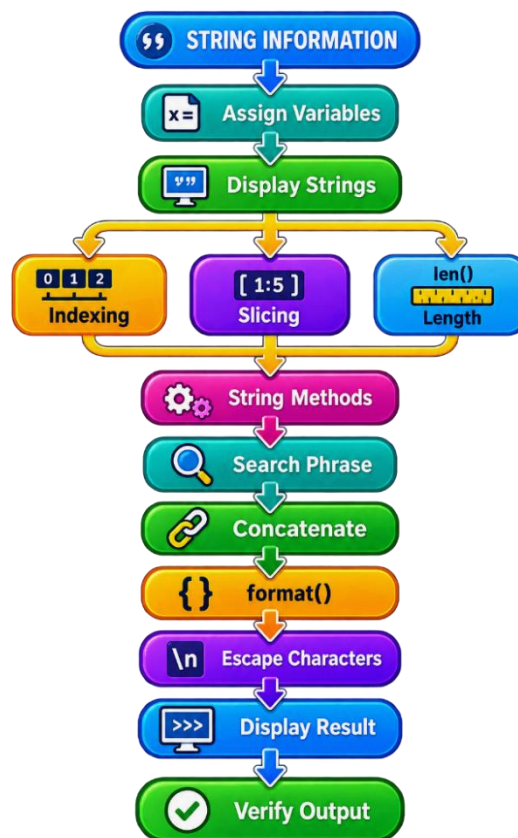


Figure 7: String Information Processing Program

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python Programming Software 3.x	Copy	1
4	Python IDLE/IDE	Software	1
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Book	Copy	1
7	Python String Exercise Files	Set	1
8	Programming Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze program requirements	Required string operations are identified
2	Open Python IDE	Development environment opens successfully
3	Create source file	New Python program is created
4	Create string variables	Correct string syntax is used
5	Display strings	print() works correctly
6	Create multiline string	Triple quotation marks are correctly applied
7	Access characters	Correct indexes are used
8	Apply positive indexing	Required characters are retrieved
9	Apply negative indexing	Characters from end are retrieved
10	Apply slicing	Required substrings are extracted
11	Determine string length	len() returns correct result
12	Apply string methods	Required string manipulation is performed
13	Search for phrase	in/not in produces correct Boolean result
14	Search character	Required character is correctly checked
15	Concatenate strings	Strings are combined correctly
16	Concatenate s and t	favorite is produced
17	Apply format()	Formatted information is displayed correctly
18	Apply escape characters	Formatted text output is correct
19	Save program	.py extension is used
20	Execute program	Program runs successfully
21	Check program output	Actual output is reviewed

No.	Procedure/Step	Key Performance
22	Correct identified errors	Program is corrected as required
23	Re-execute program	Corrected program runs
24	Verify final output	Output matches requirements

Program Requirements

Part 1 — Create String Variables

Create:

```
student_name = "Dara Sok"
course = "Python Programming"
institution = "Technical Training Center"
```

Part 2 — Create a Multiline String

Create a multiline course description.

Example:

```
description = """Python Programming Course
Learning Outcome: String Handling
Level: II"""
```

Part 3 — Display Information

Display all string variables using print().

Part 4 — Access Characters

Display:

- First character of student name.
- Second character of student name.
- Last character of student name.
- First character of course.
- Last character of course.

Part 5 — Slice Strings

Extract:

```
Python
Programming
```

from:

```
Python Programming
using slicing.
```

Part 6 — Determine Length

Display:

```
len(student_name)
len(course)
```

Part 7 — Apply String Methods

Display the course using:

```
upper()
lower()
title()
```

Use `replace()` to replace:

Python

with:

AI Python

Part 8 — Search String

Check whether:

Python

exists in the course.

Check whether:

Java

exists in the course.

Part 9 — Required Concatenation

Create:

```
s = "favo"
```

```
t = "rite"
```

Concatenate:

```
favorite = s + t
```

Display:

```
favorite
```

Part 10 — Format Information

Use `format()` to display:

```
Student Dara Sok is studying Python Programming.
```

Part 11 — Escape Characters

Display a student information header using `\t`.

Example:

```
print("Name\tCourse")
```

```
print("Dara Sok\tPython Programming")
```

Display multiple lines using `\n`.

Example Program

```
# Student String Information Processing Program
```

```
student_name = "Dara Sok"
```

```
course = "Python Programming"
```

```
institution = "Technical Training Center"
```

```
description = """Python Programming Course
```

```
Learning Outcome: String Handling
```

```
Level: II"""
```

```
print("STUDENT INFORMATION")
```

```
print("Name:", student_name)
```

```
print("Course:", course)
```

```
print("Institution:", institution)
```

```
print("\nCOURSE DESCRIPTION")
```

```
print(description)
```

```

print("\nSTRING INDEXING")
print("First character:", student_name[0])
print("Second character:", student_name[1])
print("Last character:", student_name[-1])

print("\nSTRING SLICING")
print(course[0:6])
print(course[7:])

print("\nSTRING LENGTH")
print("Student name length:", len(student_name))
print("Course length:", len(course))

print("\nSTRING METHODS")
print("Upper:", course.upper())
print("Lower:", course.lower())
print("Title:", course.title())
print("Replace:", course.replace("Python", "AI Python"))

print("\nSTRING SEARCH")
print("Python" in course)
print("Java" in course)

print("\nSTRING CONCATENATION")
s = "favo"
t = "rite"
print(s + t)

print("\nSTRING FORMAT")
print("Student {} is studying {}".format(student_name, course))

print("\nESCAPE CHARACTERS")
print("Name\tCourse")
print("{}\t{}".format(student_name, course))
print("Python\nProgramming")

```

Example Expected Output

STUDENT INFORMATION

Name: Dara Sok
 Course: Python Programming
 Institution: Technical Training Center

COURSE DESCRIPTION

Python Programming Course
 Learning Outcome: String Handling
 Level: II

STRING INDEXING

First character: D
Second character: a
Last character: k

STRING SLICING

Python
Programming

STRING LENGTH

Student name length: 8
Course length: 18

STRING METHODS

Upper: PYTHON PROGRAMMING
Lower: python programming
Title: Python Programming
Replace: AI Python Programming

STRING SEARCH

True
False

STRING CONCATENATION

favorite

STRING FORMAT

Student Dara Sok is studying Python Programming.

ESCAPE CHARACTERS

Name Course
Dara Sok Python Programming
Python
Programming

WORKSHEET No. 4.1

Title: Python String Handling Practice Worksheet

Trainee Name: _____

Date: _____

Part A — Identify Strings

Identify whether each value is a Python string.

Value	String? Yes/No
"Python"	_____
20	_____
'Dara '	_____
85.5	_____
"20"	_____
True	_____

Part B — Indexing

Given:

```
word = "Python"
```

Complete:

Expression	Result
word[0]	_____
word[1]	_____
word[2]	_____
word[5]	_____
word[-1]	_____
word[-2]	_____

Part C — Slicing

Given:

```
course = "Python Programming"
```

Complete:

Expression	Result
course[0:6]	_____
course[:6]	_____
course[7:]	_____
course[-11:]	_____

Part D — String Length

Determine:

```
len("Python")
```

Answer: _____

```
len("Hello")
```

Answer: _____

```
len("Hello World")
```

Answer: _____

Part E — String Methods

Given:

```
text = "Python Programming"
```

Complete:

Method	Result
text.upper()	_____
text.lower()	_____
text.title()	_____
text.replace("Python", "AI")	_____
text.find("Python")	_____

Part F — Search

Given:

```
course = "Python Programming"
```

Determine:

Expression	Result
"Python" in course	_____
"Java" in course	_____
"P" in course	_____
"Java" not in course	_____

Part G — Concatenation

Given:

```
s = "favo"
t = "rite"
```

Write the Python statement required to produce:

favorite

Answer:

Part H — format()

Given:

```
name = "Dara"
course = "Python"
```

Use format() to produce:

Dara studies Python.

Answer

Part I — Escape Characters

Write the appropriate escape sequence.

Requirement	Escape Sequence
New line	_____
Tab	_____
Backslash	_____
Double quotation mark	_____

PERFORMANCE CHECKLIST No. 4.1

Title: String Handling in Python

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Identified Python strings correctly	☐	☐
2	Explained basic string characteristics	☐	☐
3	Applied correct string syntax	☐	☐
4	Created strings using single or double quotation marks	☐	☐
5	Displayed strings using print()	☐	☐
6	Assigned string values to variables	☐	☐
7	Created a multiline string	☐	☐
8	Used triple quotation marks correctly	☐	☐
9	Identified string character positions	☐	☐
10	Accessed individual characters using indexing	☐	☐
11	Applied positive indexing correctly	☐	☐
12	Applied negative indexing correctly	☐	☐
13	Applied string slicing correctly	☐	☐
14	Applied negative slicing correctly	☐	☐
15	Determined string length using len()	☐	☐
16	Applied upper() correctly	☐	☐
17	Applied lower() correctly	☐	☐
18	Applied other required string methods	☐	☐
19	Searched for a phrase within a string	☐	☐
20	Searched for a character within a string	☐	☐
21	Applied in correctly	☐	☐
22	Applied not in correctly	☐	☐
23	Concatenated strings using +	☐	☐
24	Assigned s = "favo" correctly	☐	☐
25	Assigned t = "rite" correctly	☐	☐
26	Concatenated s and t to produce favorite	☐	☐
27	Applied the format() method correctly	☐	☐

No.	Criteria	Yes	No
28	Used placeholders correctly	☐	☐
29	Applied newline escape character \n	☐	☐
30	Applied tab escape character \t	☐	☐
31	Applied quotation-mark escape characters	☐	☐
32	Applied backslash escape character	☐	☐
33	Recognized that Python strings are immutable	☐	☐
34	Created and saved the Python program with .py extension	☐	☐
35	Executed the Python program successfully	☐	☐
36	Checked program output	☐	☐
37	Corrected basic string-handling errors	☐	☐
38	Verified the final output against requirements	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____